

you might want to modify the text in the file in some way. For example, you might want to read through a file of weather data and work with any line that includes the word *sunny* in the description of that day's weather. In a news report, you might look for any line with the tag <headline> and rewrite that line with a specific kind of formatting.

You can use the `splitlines()` method to turn a long string into a set of lines, and then use a `for` loop to examine each line from a file, one at a time:

```
file_reader.py from pathlib import Path

path = Path('pi_digits.txt')
❶ contents = path.read_text()

❷ lines = contents.splitlines()
  for line in lines:
    print(line)
```

We start out by reading the entire contents of the file, as we did earlier ❶. If you're planning to work with the individual lines in a file, you don't need to strip any whitespace when reading the file. The `splitlines()` method returns a list of all lines in the file, and we assign this list to the variable `lines` ❷. We then loop over these lines and print each one:

```
3.1415926535
  8979323846
  2643383279
```

Since we haven't modified any of the lines, the output matches the original text file exactly.

Working with a File's Contents

After you've read the contents of a file into memory, you can do whatever you want with that data, so let's briefly explore the digits of *pi*. First, we'll attempt to build a single string containing all the digits in the file with no whitespace in it:

```
pi_string.py from pathlib import Path

path = Path('pi_digits.txt')
contents = path.read_text()

lines = contents.splitlines()
pi_string = ''
❶ for line in lines:
    pi_string += line

print(pi_string)
print(len(pi_string))
```

We start by reading the file and storing each line of digits in a list, just as we did in the previous example. We then create a variable, `pi_string`,